

Лабораторная работа №5

Содержание

1. Цель работы 🎯	3
2. Теоретические сведения 📖	4
3. Порядок выполнения работы 📋	5
3.1. Создание проекта	5
3.2. Структура проекта и назначение файлов	5
3.3. Установка NuGet пакетов	5
4. Исходный код и комментарии ⚙️	7
4.1. Product.cs — модель данных Product	7
4.2. StoreDbContext.cs — контекст базы данных	7
4.3. Program.cs — конфигурация приложения	8
4.4. ProductController.cs — контроллер товаров	9
4.5. OrderController.cs — контроллер заказов	11
5. Тестирование приложения 🔍	13
6. Контрольные вопросы ❓	16

List of Listings

4.1	Модель данных <code>Product.cs</code>	7
4.2	Контекст базы данных <code>StoreDbContext.cs</code>	8
4.3	Конфигурация приложения <code>Program.cs</code>	9
4.4	Контроллер <code>ProductController.cs</code>	10
4.5	Контроллер <code>OrderController.cs</code>	12
5.1	Пример JSON для POST запроса (Товар 1)	14
5.2	Пример JSON для POST запроса (Товар 2)	14
5.3	Пример ответа на GET <code>/api/Product</code>	15

1. Цель работы

Целью настоящей лабораторной работы является освоение базовых принципов создания RESTful-сервиса с применением технологии ASP.NET Core Web API и базы данных SQLite. В ходе выполнения работы осуществляется реализация функциональности добавления и получения информации о товарах, а также автоматическая генерация документации с использованием Swagger.

Задание:

1. Изучить структуру ASP.NET Core Web API-проекта.
2. Настроить Entity Framework Core и контекст базы данных.
3. Создать модели и реализовать связи между ними.
4. Реализовать контроллеры для управления товарами и заказами.
5. Обеспечить создание базы данных при первом запуске.

2. Теоретические сведения

ASP.NET Core Web API представляет собой инфраструктуру для разработки HTTP-сервисов, архитектура которых основывается на контроллерах и маршрутах. Это позволяет обрабатывать REST-запросы и предоставлять данные в формате JSON.

Ключевые технологии:

- ASP.NET Core
- Entity Framework Core
- SQLite
- REST

Компоненты проекта:

- `Models` — классы бизнес-логики и структуры таблиц.
- `Data` — контекст базы данных.
- `Controllers` — обработка HTTP-запросов.

3. Порядок выполнения работы №

В данном разделе последовательно описан процесс подготовки, запуска и анализа исходного кода проекта.

3.1. Создание проекта

Для начала необходимо выполнить следующие действия:

1. В среде Visual Studio выбрать шаблон "ASP.NET Core Web API".
2. Указать имя проекта: `StoreManagerApi`.
3. При создании проекта активировать следующие опции:
 - Controllers (включает шаблон контроллера);
 - Enable OpenAPI support (включает Swagger);
 - Enable HTTPS (в случае технических проблем допускается переключение на HTTP).

3.2. Структура проекта и назначение файлов

Структура проекта выглядит следующим образом:

```
StoreManagerApi/
├─ Controllers/
│   └─ ProductController.cs // Обрабатывает HTTP-запросы, связанные с товарами
├─ Data/
│   └─ StoreDbContext.cs   // Представляет контекст базы данных SQLite
├─ Models/
│   └─ Product.cs          // Описывает модель данных для сущности "Товар"
├─ Program.cs              // Конфигурирует сервисы и middleware приложения
└─ appsettings.json        // Хранит параметры конфигурации
```

3.3. Установка NuGet пакетов

Необходимо выполнить Tools → NuGet Package Manager → Package Manager Console (или кликнуть правой кнопкой мыши на решении и выбрать «Управление пакетами NuGet для решения»). Затем ввести названия требующихся пакетов:

- Microsoft.EntityFrameworkCore (если не установлен автоматически)

- `Microsoft.EntityFrameworkCore.Sqlite`
- `Microsoft.EntityFrameworkCore.Tools`
- `Swashbuckle.AspNetCore` (для документации Swagger, если не установлен автоматически)

Для обеспечения работы всех необходимых функций в проекте необходимо установить указанные NuGet-пакеты.

4. Исходный код и комментарии

В данном разделе рассмотрены ключевые компоненты проекта, реализующие бизнес-логику и взаимодействие с HTTP-запросами.

4.1. Product.cs — модель данных Product

```
1 namespace StoreManagerApi.Models
2 {
3     // Модель отражает структуру таблицы "Products" в базе данных
4     public class Product
5     {
6         public int Id { get; set; } // Первичный ключ (идентификатор товара)
7         public string Name { get; set; } // Название товара
8         public decimal Price { get; set; } // Цена товара
9         public int Stock { get; set; } // Количество на складе
10        public string Category { get; set; } // Категория товара
11    }
12 }
```

Листинг 4.1 – Модель данных Product.cs

В листинге 4.1 представлена модель Product, отражающая свойства таблицы «Товары» в базе данных. В Entity Framework Core классы, представляющие сущности, называются моделями. Они определяют структуру таблиц в базе данных. Каждое свойство класса соответствует столбцу таблицы.

4.2. StoreDbContext.cs — контекст базы данных

```
1 using Microsoft.EntityFrameworkCore;
2 using StoreManagerApi.Models;
3
4 namespace StoreManagerApi.Data
5 {
6     // Класс контекста базы данных, используемый Entity Framework Core
7     public class StoreDbContext : DbContext
8     {
9         // Конструктор с передачей параметров подключения
10        public StoreDbContext(DbContextOptions<StoreDbContext> options) : base(options) { }
11
12        // Наборы данных (таблицы)
13        public DbSet<Product> Products => Set<Product>();
14        public DbSet<Order> Orders => Set<Order>();
15        public DbSet<OrderItem> OrderItems => Set<OrderItem>();
16
17        // Конфигурация модели при создании схемы базы данных
```

```
18     protected override void OnModelCreating(ModelBuilder modelBuilder)
19     {
20         modelBuilder.Entity<Order>()
21             .HasMany(o => o.Items)      // Один заказ содержит много позиций
22             .WithOne(oi => oi.Order)    // Обратная навигация
23             .HasForeignKey(oi => oi.OrderId) // Указан внешний ключ
24             .OnDelete(DeleteBehavior.Cascade); // При удалении заказа – удалить все
25             // ↳ позиции
26
27         modelBuilder.Entity<OrderItem>()
28             .HasOne(oi => oi.Product)    // Каждая позиция ссылается на один продукт
29             .WithMany()                  // Один продукт может быть в нескольких позициях
30             .HasForeignKey(oi => oi.ProductId) // Внешний ключ на продукт
31             .OnDelete(DeleteBehavior.Restrict); // Запретить удаление продукта, если он в
32             // ↳ заказах
33     }
```

Листинг 4.2 – Контекст базы данных [StoreDbContext.cs](#)

Класс `DbContext` (Листинг 4.2) является ключевым компонентом EF Core. Он управляет доступом к базе данных и позволяет выполнять операции CRUD (Create, Read, Update, Delete).

4.3. Program.cs — конфигурация приложения

```
1  using Microsoft.EntityFrameworkCore;
2  using StoreManagerApi.Data;
3
4  var builder = WebApplication.CreateBuilder(args);
5
6  // Добавление контекста базы данных и конфигурация SQLite
7  builder.Services.AddDbContext<StoreDbContext>(options =>
8      options.UseSqlite("Data Source=store.db")); // Строка подключения
9
10 builder.Services.AddControllers(); // Добавление контроллеров
11 builder.Services.AddEndpointsApiExplorer(); // Для Swagger
12 builder.Services.AddSwaggerGen(); // Подключение Swagger-документации
13
14 var app = builder.Build();
15
16 // === Создание БД при старте (если нет) ===
17 using (var scope = app.Services.CreateScope())
18 {
19     var db = scope.ServiceProvider.GetRequiredService<StoreDbContext>();
20     db.Database.EnsureCreated();
21 }
22
23 if (app.Environment.IsDevelopment())
```



```
24 {
25     app.UseSwagger();    // Включение Swagger
26     app.UseSwaggerUI();  // UI интерфейс Swagger
27 }
28
29 app.UseHttpsRedirection(); // Перенаправление на HTTPS
30
31 // app.UseAuthorization(); // Отключено, если нет аутентификации
32
33 app.MapControllers();    // Подключение маршрутов контроллеров
34
35 app.Run();               // Запуск приложения
```

Листинг 4.3 – Конфигурация приложения [Program.cs](#)

Файл Program.cs (Листинг 4.3) отвечает за полную конфигурацию среды исполнения Web API.

1. AddDbContext(...) — добавляет и настраивает контекст EF Core для работы с базой данных SQLite.
2. EnsureCreated() — при первом запуске создает файл базы данных, если он отсутствует.
3. AddSwaggerGen() — подключает автоматическую генерацию документации Swagger.
4. MapControllers() — включает маршрутизацию и обработку HTTP-запросов контроллерами.
5. UseHttpsRedirection() — автоматически перенаправляет все запросы на HTTPS.
6. app.Run() — завершает настройку и запускает веб-сервер приложения.

Методы Use... и Map... добавляют middleware-компоненты в конвейер обработки запросов.

4.4. ProductController.cs — контроллер товаров

```
1 using Microsoft.AspNetCore.Mvc;
2 using Microsoft.EntityFrameworkCore;
3 using StoreManagerApi.Data;           // Подключение контекста базы данных
4 using StoreManagerApi.Models;        // Подключение модели Product
5
6 namespace StoreManagerApi.Controllers;
7
8 // Атрибут указывает, что это API-контроллер
9 [ApiController]
10
11 // Базовый маршрут: api/Product
12 [Route("api/[controller]")]
13 public class ProductController : ControllerBase
14 {
15     // Внедрение зависимости – контекст базы данных
```

```
16 private readonly StoreDbContext _context;
17
18 // Конструктор контроллера, получающий контекст через DI (Dependency Injection)
19 public ProductController(StoreDbContext context) => _context = context;
20
21 // Метод GET: Возвращает все товары
22 [HttpGet]
23 public async Task<ActionResult<IEnumerable<Product>>> GetAll() =>
24     await _context.Products.ToListAsync(); // Асинхронное извлечение всех записей
25
26 // Метод GET: Возвращает один товар по ID
27 [HttpGet("{id}")]
28 public async Task<ActionResult<Product>> Get(int id)
29 {
30     var product = await _context.Products.FindAsync(id); // Поиск товара
31     return product == null ? NotFound() : Ok(product); // Если не найден – 404
32 }
33
34 // Метод POST: Создаёт новый товар
35 [HttpPost]
36 public async Task<ActionResult> Create(Product product)
37 {
38     _context.Products.Add(product); // Добавление нового объекта
39     await _context.SaveChangesAsync(); // Сохранение изменений
40     return CreatedAtAction(nameof(Get), // Возврат 201 Created
41         new { id = product.Id }, product);
42 }
43
44 // Метод PUT: Обновляет товар по ID
45 [HttpPut("{id}")]
46 public async Task<ActionResult> Update(int id, Product product)
47 {
48     if (id != product.Id) return BadRequest(); // Проверка ID
49
50     _context.Entry(product).State = EntityState.Modified; // Помечаем как изменённый
51     await _context.SaveChangesAsync(); // Сохраняем изменения
52     return NoContent(); // Возврат 204 No Content
53 }
54
55 // Метод DELETE: Удаляет товар по ID
56 [HttpDelete("{id}")]
57 public async Task<ActionResult> Delete(int id)
58 {
59     var product = await _context.Products.FindAsync(id); // Поиск объекта
60     if (product == null) return NotFound(); // 404 если не найден
61
62     _context.Products.Remove(product); // Удаление объекта
63     await _context.SaveChangesAsync(); // Сохранение изменений
64     return NoContent(); // Возврат 204 No Content
65 }
66 }
```

Листинг 4.4 – Контроллер `ProductController.cs`

Контроллер `ProductController` (Листинг 4.4) предназначен для управления сущностями товаров

через HTTP-запросы. Он реализует CRUD операции. Основные задачи:

1. Получение всех товаров: GET /api/Product.
2. Получение одного товара по ID: GET /api/Product/id.
3. Создание нового товара: POST /api/Product.
4. Редактирование товара: PUT /api/Product/id.
5. Удаление товара: DELETE /api/Product/id.

4.5. OrderController.cs — контроллер заказов

```
1 using Microsoft.AspNetCore.Mvc;
2 using Microsoft.EntityFrameworkCore;
3 using StoreManagerApi.Data;           // Подключение контекста базы данных
4 using StoreManagerApi.Models;         // Подключение моделей заказа
5
6 namespace StoreManagerApi.Controllers;
7
8 [ApiController]                       // Атрибут указывает, что это API-контроллер
9 [Route("api/[controller]")]           // Базовый маршрут: api/Order
10 public class OrderController : ControllerBase
11 {
12     private readonly StoreDbContext _context; // Внедрение зависимости
13
14     public OrderController(StoreDbContext context) => _context = context; // Конструктор
15
16     [HttpPost] // Метод POST: Создаёт новый заказ
17     public async Task<ActionResult> Create(Order order)
18     {
19         if (order == null || order.Items == null || !order.Items.Any())
20             return BadRequest("Заказ пуст или не содержит товаров."); // Проверка
21
22         order.OrderDate = DateTime.Now; // Установка даты заказа
23
24         foreach (var item in order.Items)
25         {
26             item.Product = null; // Удаление вложенных объектов
27             item.Order = null;
28         }
29
30         _context.Orders.Add(order); // Добавление заказа
31         await _context.SaveChangesAsync(); // Сохранение изменений
32
33         return CreatedAtAction(nameof(GetById), new { id = order.Id }, order); // Возврат 201
34     }
35 }
```

```
36 [HttpGet] // Метод GET: Возвращает все заказы
37 public async Task<ActionResult<IEnumerable<Order>>> Get()
38 {
39     var orders = await _context.Orders
40         .Include(o => o.Items)           // Загрузка позиций заказа
41         .ThenInclude(i => i.Product)      // Загрузка информации о товаре
42         .ToListAsync();
43
44     return orders; // Возврат списка заказов
45 }
46
47 [HttpGet("{id}")] // Метод GET: Возвращает заказ по ID
48 public async Task<ActionResult<Order>> GetById(int id)
49 {
50     var order = await _context.Orders
51         .Include(o => o.Items)           // Загрузка позиций
52         .ThenInclude(i => i.Product)      // Загрузка продуктов
53         .FirstOrDefaultAsync(o => o.Id == id);
54
55     return order == null ? NotFound() : Ok(order); // 404 если не найден
56 }
57 }
```

Листинг 4.5 – Контроллер `OrderController.cs`

Контроллер `OrderController` (Листинг 4.5) отвечает за создание и вывод заказов. Описание методов:

1. `Create(Order order)` — создаёт новый заказ, устанавливает дату и сохраняет его в базе данных.
2. `Get()` — возвращает список всех заказов, включая связанные позиции и товары.
3. `GetById(int id)` — возвращает заказ по ID с полной структурой.

5. Тестирование приложения 68

При запуске приложения открывается Swagger UI — страница, где можно тестировать API.

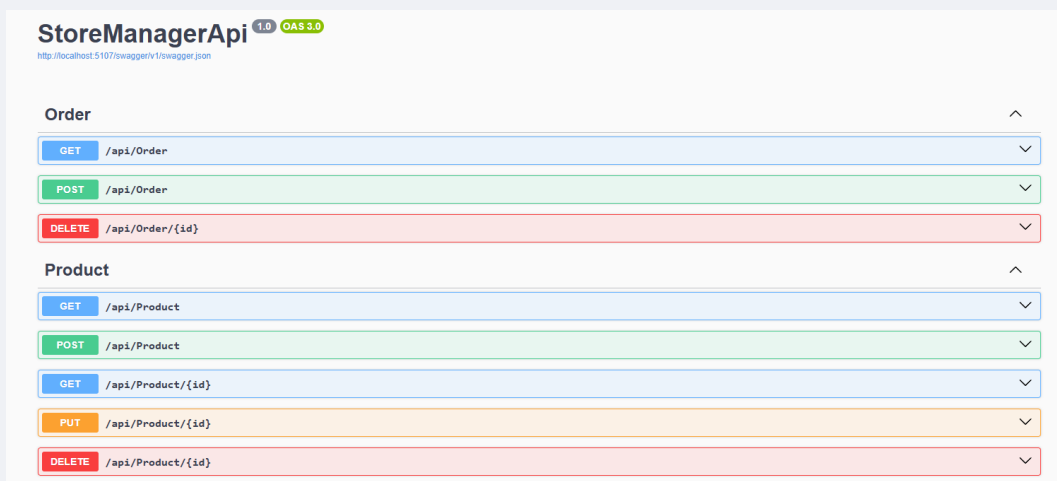


Рисунок 5.1 – Страница тестирования API Swagger UI.

Внимание: В случае ошибки "Подключение не защищено"/ "NET::ERR_CERT_INVALID в Visual Studio рядом с кнопкой запуска выберите http-профиль вместо https.

Далее необходимо протестировать методы GET и POST. С помощью метода POST можно добавить новые товары в базу данных. Для этого в разделе Product рядом с методом POST нажмите на «стрелку» и в открывшейся вкладке кнопку «Try it out».

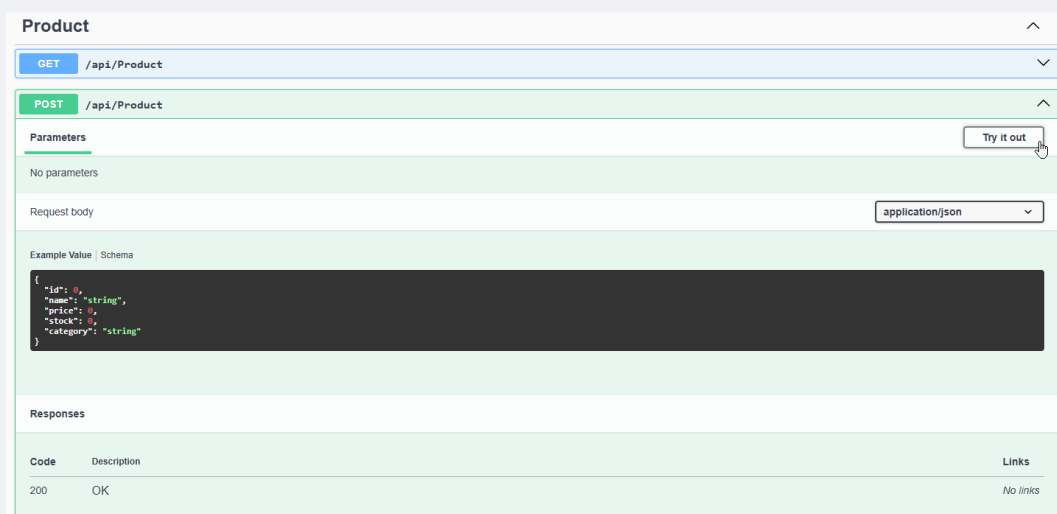


Рисунок 5.2 – Меню метода POST в Swagger UI для контроллера Product.

В открывшемся окне необходимо изменить данные JSON-объекта на требуемые и нажать кнопку «Execute».

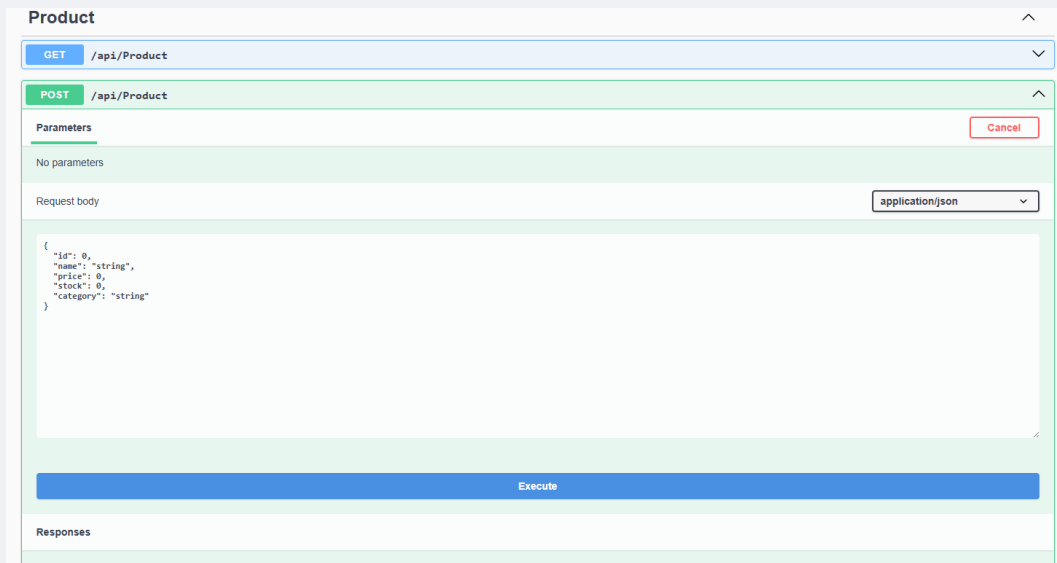


Рисунок 5.3 – Добавление данных с помощью метода POST в Swagger UI.

Пример 1: POST — Добавление товара №1

```

1 {
2   "id": 0, // id будет присвоен автоматически базой данных
3   "name": "Монитор Samsung",
4   "price": 25000,
5   "stock": 30,
6   "category": "Мониторы и дисплеи"
7 }

```

Листинг 5.1 – Пример JSON для POST запроса (Товар 1)

Пример 2: POST — Добавление товара №2

```

1 {
2   "id": 0, // id будет присвоен автоматически базой данных
3   "name": "Компьютерная мышь Logitech",
4   "price": 25.99,
5   "stock": 10,
6   "category": "Периферия"
7 }

```

Листинг 5.2 – Пример JSON для POST запроса (Товар 2)

Далее аналогичным образом можно получить данные из базы данных с помощью метода GET. Для этого в разделе GET /api/Product необходимо нажать кнопку «Try it out». В результате на экран будет выведен результат GET-запроса.

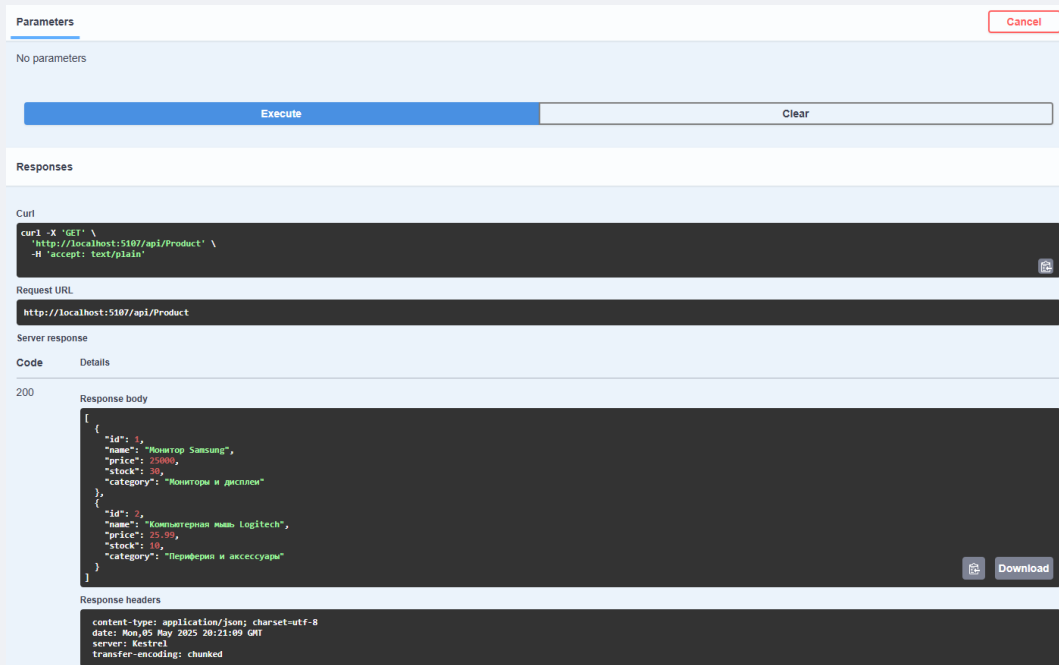


Рисунок 5.4 – Выполнение запроса GET /api/Product и его результат в Swagger UI.

Пример ответа на GET-запрос:

```

1  [
2    {
3      "id": 1,
4      "name": "Монитор Samsung",
5      "price": 25000,
6      "stock": 30,
7      "category": "Мониторы и дисплеи"
8    },
9    {
10     "id": 2,
11     "name": "Компьютерная мышь Logitech",
12     "price": 25.99,
13     "stock": 10,
14     "category": "Периферия"
15   }
16 ]

```

Листинг 5.3 – Пример ответа на GET /api/Product

Внимание: если вы столкнулись с тем, что новые изменения, которые вы вносите в приложение не работают или не дают нужного результата (программа выдает ошибку 401, 500 и т.д.) необходимо удалить файл базы данных `store.db`. Программа автоматически создаст файл заново, вам нужно будет только добавить новые товары через интерфейс приложения WPF (или Swagger UI для API).

6. Контрольные вопросы ?

1. Что такое ASP.NET Core Web API и для чего он используется?

ASP.NET Core Web API — это фреймворк для создания HTTP-сервисов, которые могут быть доступны различным клиентам (веб-браузеры, мобильные приложения и др.). Он используется для построения RESTful API, предоставляющих доступ к данным и бизнес-логике приложения.

2. Какую роль выполняет файл Program.cs в ASP.NET Core приложении?

Файл Program.cs является точкой входа приложения. Он отвечает за конфигурацию сервисов приложения (включая внедрение зависимостей, например, DbContext), настройку конвейера обработки HTTP-запросов (middleware), определение источников конфигурации и запуск веб-сервера.

3. Что такое модель (Model) в контексте EF Core и ASP.NET Core? Приведите пример из лабораторной работы.

Модель представляет структуру данных, с которыми работает приложение. В EF Core модель обычно соответствует таблице в базе данных. В ASP.NET Core модели также используются для передачи данных между контроллером и клиентом. Пример из работы: класс Product с свойствами Id, Name, Price, Stock, Category.

4. Объясните назначение класса DbContext (на примере StoreDbContext) в Entity Framework Core.

Класс DbContext (например, StoreDbContext) представляет сессию с базой данных и позволяет запрашивать и сохранять данные. Он содержит свойства типа DbSet<TEntity> для каждой сущности (таблицы) в модели. В методе OnConfiguring настраивается подключение к БД, а в OnModelCreating — детали модели данных, такие как связи и ограничения.

5. Что такое контроллер в ASP.NET Core Web API (на примере ProductController)? Какие задачи он решает?

Контроллер — это класс, который обрабатывает входящие HTTP-запросы, выполняет соответствующую бизнес-логику (часто взаимодействуя с моделями, сервисами и DbContext) и возвращает HTTP-ответы. ProductController в данной работе управляет операциями CRUD (создание, чтение, обновление, удаление) для сущностей товаров.

6. Какие HTTP-методы были реализованы в ProductController и каково их стандартное назначение в REST API?

Были реализованы:

- GET: для получения ресурсов (всех товаров или одного товара по его идентификатору).

- POST: для создания нового ресурса (нового товара).
- PUT: для полного обновления существующего ресурса (обновления информации о товаре).
- DELETE: для удаления ресурса (удаления товара по его идентификатору).

7. Что такое Swagger (OpenAPI) и для чего он использовался в лабораторной работе?

Swagger (реализация спецификации OpenAPI) — это набор инструментов для проектирования, создания, документирования и использования RESTful веб-сервисов. В лабораторной работе он использовался для автоматической генерации интерактивной документации API, которая позволяет просматривать доступные эндпоинты, их параметры, типы возвращаемых данных и тестировать их непосредственно из браузера.

8. Как в Program.cs была настроена работа с базой данных SQLite и её автоматическое создание?

Работа с SQLite была настроена путем регистрации `StoreDbContext` в контейнере зависимостей с помощью метода `builder.Services.AddDbContext<StoreDbContext>(options => options.UseSqlite("Data Source=store.db"))`; Автоматическое создание базы данных (если она не существует) при запуске приложения обеспечивалось вызовом `db.Database.EnsureCreated()`; внутри блока, получающего сервис `StoreDbContext` из провайдера сервисов.

9. Объясните принцип маршрутизации в ASP.NET Core Web API на примере ProductController.

Маршрутизация определяет, как HTTP-запросы сопоставляются с методами действий контроллера. Атрибут `[ApiController]` на классе контроллера активирует стандартное поведение для API. Атрибут `[Route("api/[controller]")]` на `ProductController` задает базовый маршрут как `/api/Product`. Далее, атрибуты HTTP-методов (`[HttpGet]`, `[HttpPost]`, `[HttpGet("id")]` и т.д.) на методах действий уточняют путь и HTTP-метод для каждого эндпоинта. Например, `[HttpGet("id")]` на методе `Get(int id)` означает, что запрос `GET /api/Product/5` будет обработан этим методом.

10. В чем заключается механизм Dependency Injection (DI), использованный в конструкторе ProductController(StoreDbContext context)?

Dependency Injection (внедрение зависимостей) — это паттерн проектирования, при котором зависимости объекта (например, экземпляр `StoreDbContext` для `ProductController`) предоставляются ему извне (обычно контейнером DI), а не создаются им самим. В `Program.cs` `StoreDbContext` регистрируется как сервис. Когда ASP.NET Core создает экземпляр `ProductController`, контейнер DI автоматически «внедряет» (передает) зарегистрированный экземпляр `StoreDbContext` в его конструктор. Это способствует слабой связанности компонентов и улучшает тестируемость.